CHAPITRE

Classes Abstraites et Interfaces

Objectifs d'apprentissage

- Distinguer les concepts de **classes abstraites** (modèles) et d'**interfaces** (contrats).
- Maîtriser l'héritage multiple de types via l'implémentation d'interfaces multiples.
- Concevoir des architectures flexibles et modulaires adaptées aux évolutions futures.

04

CHAPITRE 4.1

Classes Abstraites : Définition et Utilité

Une classe abstraite sert de **modèle incomplet**. Elle ne peut pas être utilisée directement pour créer des objets mais fournit une structure commune pour ses héritiers.

Non Instanciable

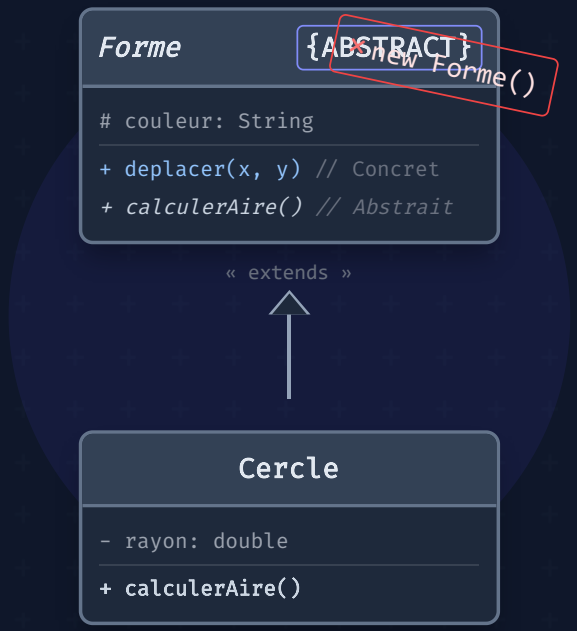
Impossible de faire `new ClasseAbstraite()`. Elle existe uniquement pour être étendue par d'autres classes.

Méthodes Abstraites

Déclare des signatures de méthodes **sans implémentation** (sans corps). Les sous-classes concrètes sont *forcées* de les implémenter.

Comportement Mixte

Peut contenir à la fois des méthodes abstraites (contrat) et des méthodes concrètes (code partagé) pour éviter la duplication.



La classe `Cercle` **doit** fournir le code pour *`calculerAire()`*.

CHAPITRE 4.1.2

Classe Abstraite (Modèle)

Forme.java

```java
// "abstract" empêche l'instanciation directe
public abstract class Forme {
  protected String couleur;
  protected double x, y;
  public Forme(String couleur, double x, double y) {
    this.couleur = couleur;
    this.x = x; this.y = y;
  }
  // Méthodes abstraites (SANS corps {})
  public abstract double calculerAire();
  public abstract double calculerPerimetre();
  // Méthode concrète (héritée telle quelle)
  public void afficherInfo() {
    System.out.println("Forme de couleur " + couleur);
  }
}
```

Sert de gabarit pour Cercle, Rectangle, etc.

Concepts Clés

Non Instanciable

`new Forme(...)` provoquera une erreur de compilation. On doit instancier une classe fille concrète (ex: `new Cercle(...)`).

Méthodes Abstraites

Définissent un "contrat". Les classes filles **doivent obligatoirement** fournir une implémentation (Override), sinon elles doivent être abstraites aussi.

Méthodes Concrètes

Permettent de factoriser du code commun (attributs, getters/setters, logique partagée) pour éviter la duplication.

Usage Typique

Utilisez une classe abstraite quand vous avez une relation "est-un" forte et du code à partager, mais que le concept parent est trop générique pour exister seul.

CHAPITRE 4.1.3

Implémentation & Usage Polymorphe

TestFormes.java

TestFormes.java

```java
class Cercle extends Forme {
  private double rayon;
  public Cercle(String c, double x, double y, double r) {
    super(c, x, y); this.rayon = r;
  }
  @Override
  public double calculerAire() {
    return Math.PI * rayon * rayon; // Implémentation spécifique
  }
  // Implémentation de calculerPerimetre() ...
}
public class TestFormes {
  public static void main(String[] args) {
    // Tableau polymorphe : type déclaré 'Forme', objets réels divers
    Forme[] formes = {
      new Cercle("rouge", 0, 0, 5),
      new Rectangle("bleu", 10, 10, 4, 6)
    };
    for (Forme f : formes) {
      f.afficherInfo(); // Appelle implicitement calculerAire() du sous-type
    }
```

Concrétisation

La classe `Cercle` n'est plus abstraite car elle fournit une implémentation pour **toutes** les méthodes abstraites héritées (`calculerAire`, etc.).

Collections Hétérogènes

Le tableau `formes` stocke des références de type parent `Forme`, mais contient en réalité des objets `Cercle` et `Rectangle`.

Invocation Dynamique

La méthode `afficherInfo()` (définie dans Forme) appelle `calculerAire()`. Au runtime, c'est la version spécifique à chaque objet (cercle ou rectangle) qui est exécutée.

CHAPITRE 4.1

Système de documents : Abstractions

Document.java

```java
public abstract class Document {
  protected String titre;
  public Document(String t) { this.titre = t; }
  // Méthode abstraite : LE CONTRAT
  public abstract void ouvrir();
  // Méthode concrète : PARTAGÉE
  public void info() {
    System.out.println("Doc: " + titre);
  }
}
```

PDF.java

```java
public class PDF extends Document {
  public PDF(String t) { super(t); }
  @Override
  public void ouvrir() {
    // Implémentation spécifique OBLIGATOIRE
    System.out.println("Lancement d'Adobe ... ");
  }
}
```

Concept de classe abstraite

Une classe `abstract` ne peut pas être instanciée (`new Document()` est interdit). Elle sert de gabarit partiel, factorisant le code commun (comme `titre` et `info()`) tout en laissant des trous à combler.

Méthodes abstraites

Les méthodes déclarées `abstract` n'ont pas de corps. Elles définissent un **contrat** : "toute sous-classe concrète DOIT savoir s'ouvrir". Le compilateur vérifie que `PDF` implémente bien `ouvrir()`.

Utilisation polymorphe

L'intérêt majeur est de manipuler des collections de documents sans connaître leur type réel :

```java
List<Document> docs = new ArrayList<>();
```

On peut appeler `doc.ouvrir()` sur chaque élément, l'exécution s'adaptera (PDF, Word, etc.).

CHAPITRE 4.2.1

Classes Abstraites vs Interfaces

Héritage Contrat

CRITÈRE	Classe Abstraite	Interface
Attributs (État)	✓ Variables d'instance autorisées (private, protected, etc.)	ⓘ Constantes uniquement `public static final`
Constructeurs	✓ Oui, pour initialiser l'état Appelé via super()	✗ Non, pas d'instanciation Pas d'état à initialiser
Méthodes	Abstraites ou Concrètes Visibilité libre (sauf private abstract)	Abstraites (public) par défaut + default, static, private (Java 9+)
Héritage	`extends` Simple (Unique)	`implements` Multiple
Usage & Sémantique	Relation **"Est un"** Partage de code et comportement commun	Capacité **"Peut faire"** Définition de contrat / API publique

CHAPITRE 4.2.3

L'Interface Volant

Volant.java & Avion.java

```java
public interface Volant {
  // Constante implicite : public static final
  double VITESSE_MAX = 1000.0;
  // Méthodes implicites : public abstract
  void decoller();
  void atterrir();
  void voler(double altitude);
}
public class Avion implements Volant {
  @Override
  public void decoller() {
    System.out.println("L'avion décolle sur la piste.");
  }
  // ... Doit implémenter atterrir() et voler()
}
```

Le mot-clé 'implements' active le contrat

Notes d'Implémentation

Le Contrat Strict

La classe Avion **s'engage** à implémenter toutes les méthodes définies dans Volant. Sinon, la classe doit être abstraite.

Modificateurs Implicites

Dans une interface, toutes les méthodes sont par défaut `public abstract` et les variables sont `public static final` (constantes).

Polymorphisme

Un objet de type Avion peut être stocké dans une variable de type Volant.

```
Volant v = new Avion();
```

Flexibilité

Contrairement à l'héritage (`extends`), une classe peut implémenter **plusieurs** interfaces séparées par des virgules.

CHAPITRE 4.2

Interfaces Multiples & Héritage

Implémentation Multiple

Canard.java

```java
public class Canard
  implements Volant, Nageant, Marchant {
  // Doit implémenter TOUTES les méthodes
  public void voler() { ... }
  public void nager() { ... }
  public void marcher() { ... }
}
```

Composition de rôles

Contrairement aux classes (héritage simple), une classe Java peut implémenter un nombre illimité d'interfaces. C'est un moyen puissant de définir qu'un objet possède **plusieurs capacités** distinctes.

```
is-a Canard & has-capabilities [Volant, Nageant...]
```

Héritage d'Interfaces

Vehicules.java

```java
interface Vehicule {
  void demarrer();
}
// Extension de contrat
interface VehiculeElec extends Vehicule {
  void charger();
}
```

Extension de contrat

Une interface peut hériter d'une ou plusieurs autres interfaces avec `extends`. La classe qui implémente l'interface fille doit fournir le code pour les méthodes de **toute la hiérarchie**.

```
Tesla implements VehiculeElec => demarrer() + charger()
```

CHAPITRE 4.2.6

Interfaces Fonctionnelles & Lambdas

TestLambda.java

```java
@FunctionalInterface
public interface Calculable {
  double calculer(double a, double b);
  // max 1 méthode abstraite
}
public class TestApp {
  public static void main(String[] args) {
    // Implémentation via Lambda (Java 8+)
    Calculable add = (a, b) → a + b;
    Calculable mul = (a, b) → a * b;
    // Utilisation
    System.out.println(add.calculer(5, 3)); // 8.0
  }
}
```

Sortie: 8.0

Concepts Clés (Java 8+)

1

Règle Unique

Une interface fonctionnelle doit contenir **exactement une méthode abstraite**. Elle peut avoir plusieurs méthodes `default` ou `static`.

Expression Lambda

Une syntaxe concise pour implémenter ces interfaces "à la volée", évitant les classes anonymes verbeuses.

Annotation de sécurité

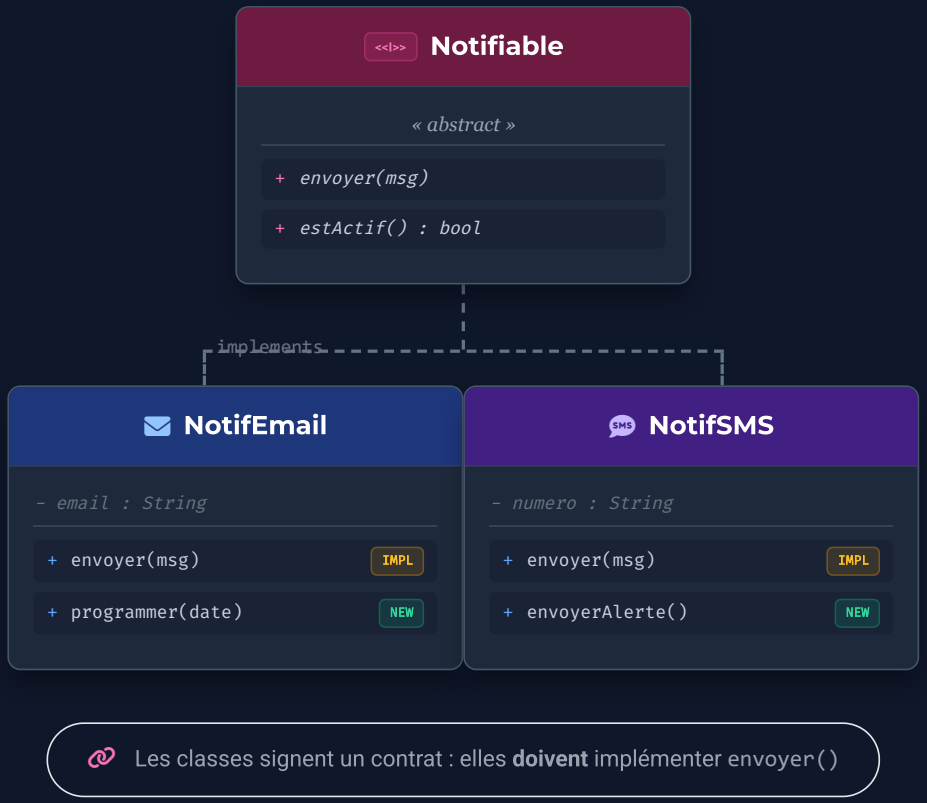
`@FunctionalInterface` est optionnelle mais recommandée : elle permet au compilateur de vérifier la règle de la méthode unique.

Bibliothèque Standard

Java fournit déjà de nombreuses interfaces fonctionnelles prêtes à l'emploi dans `java.util.function` (Predicate, Consumer, Function, Supplier...).

INTERFACES

Système de Notification



Les classes signent un contrat : elles **doivent** implémenter `envoyer()`

Avantages de l'Interface

Pourquoi utiliser une interface plutôt qu'une classe concrète ?

Contrat Strict

L'interface `Notifiable` garantit que tout objet compatible possède la méthode `envoyer()`, quelle que soit son implémentation interne.

Découplage Fort

Le gestionnaire de notifications ne connaît que l'interface. On peut ajouter `NotifPush` ou `NotifSlack` sans modifier le code existant.

Héritage Multiple (Type)

Une classe comme `NotifEmail` pourrait implémenter à la fois `Notifiable` et `Archivable`, ce qui est impossible avec l'héritage de classes simple.